

DNN ASSIGNMENT Group No 163

Group Member Names:

1. ABHIJIT DEY(2024AA05076)
2. GAURAV DHAMI (2024AA05077)
3. MANISH GARG (2024AA05118)
4. SUKAMATI NAVEEN KUMAR (2024AA05078)

1. Import the required libraries

```
In [17]: import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.datasets import cifar10
from tensorflow.keras.layers import Dropout
from tensorflow.keras import regularizers
from tensorflow.keras.optimizers import SGD, RMSprop, Adam
from keras.models import Sequential
from keras.layers import Dense, Activation, BatchNormalization
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, classification_report
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

2. Data Acquisition -- Score: 0.5 Mark

Points noted from the data set selected CIFAR10

1. We choose Cifar10 data set because it is widely used and having diverse set of colour images, making us to explore and evaluate DNN parameters in better way.
2. Data set has 60000 coloured images and 10 different classes.
3. We found the dataset to be challenging and interesting because the images are small and contain a lot of variation in backgrounds, colors, and object positions. This makes it a good test for the learning ability of deep neural networks.
4. The dataset is split into 50,000 training images and 10,000 test images, which allowed us to properly train our model and then evaluate its performance on unseen data.

2.1 Code for converting the above downloaded data into a form suitable for DL

```
In [18]: (x_train, y_train), (x_test, y_test) = cifar10.load_data()
print(f"Training samples: {x_train.shape}, Test samples: {x_test.shape}")
```

Training samples: (50000, 32, 32, 3), Test samples: (10000, 32, 32, 3)

2.1 Write your observations from the above.

1. Size of the dataset

The CIFAR-10 dataset consists of 60,000 color images in total. **Training set:** 50,000 images **Test set:** 10,000 images Each image is of size 32x32 pixels with 3 color channels RGB.

2. What type of data attributes are there?

Features: The data consists of small color images (numerical pixel values) with three channels (Red, Green, Blue), each pixel value ranging from 0 to 255. After preprocessing, these are normalized to the range 0 to 1 and flattened into vectors of length 3072 ($32 \times 32 \times 3$).

Labels: The target variable is categorical, representing one of 10 classes (e.g., airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)

3. What are you classifying?:

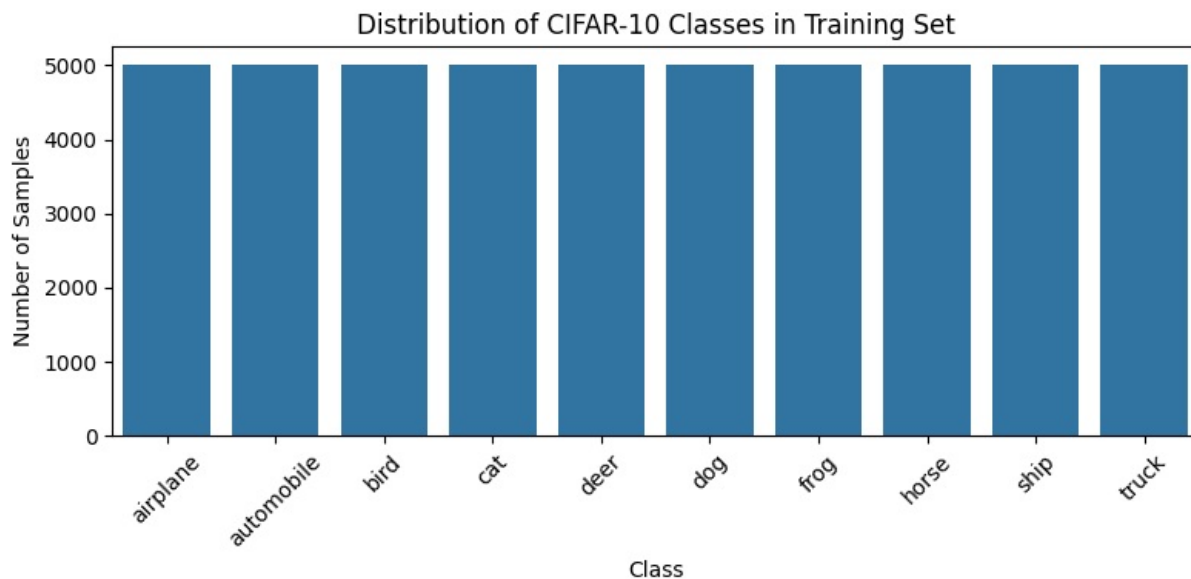
We are classifying each image into one of 10 categories. The goal is to predict the correct class label for each input image based on its pixel values

4. Plot the distribution of the categories of the target / label.

```
In [19]: # CIFAR-10 class names
class_names = [
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]

# Flatten y_train to 1D array for counting
y_train_flattened = y_train.flatten()

# Plot the distribution
plt.figure(figsize=(8, 4))
sns.countplot(x=y_train_flattened)
plt.title('Distribution of CIFAR-10 Classes in Training Set')
plt.xlabel('Class')
plt.ylabel('Number of Samples')
plt.xticks(ticks=np.arange(10), labels=class_names, rotation=45)
plt.tight_layout()
plt.show()
```



3. Data Preparation -- Score: 1 Mark

Perform the data preprocessing that is required for the data that you have downloaded.

This stage depends on the dataset that is used.

3.1 Apply pre-processing techniques

Below are the preprocessing techniques applied

1. Normalization
2. Flattening
3. One-hot Encoding

```
In [20]: # normalize the data
x_train = x_train / 255.0
x_test = x_test / 255.0
# flattening the data
x_train_flat = x_train.reshape(-1, 32 * 32 * 3)
x_test_flat = x_test.reshape(-1, 32 * 32 * 3)
print(f"Flattened shape: {x_train_flat.shape}")
```

Flattened shape: (50000, 3072)

3.2 Identify the target variables.

- Separate the data from the target such that the dataset is in the form of (X,y) or (Features, Label)
- Discretize / Encode the target variable or perform one-hot encoding on the target or any other as and if required.

```
In [21]: # Convert integer labels to one-hot vectors for multi-class classification
y_train_cat = to_categorical(y_train, 10)
y_test_cat = to_categorical(y_test, 10)
```

3.3 Split the data into training set and testing set

The size of the training and testing datasets for CIFAR-10 in our assignment is: Training dataset: 50,000 samples (images) Testing dataset: 10,000 samples (images) Splitted as 80% training data and 20% test data

3.4 Preprocessing report

- We did normalization and flattening along with categorical encoding
- We scaled pixel values to improve numerical stability and converted 3D image data to 1D vector
- converted integer labels to one-hot vectors for multi-class classification
- Data is clean and no duplicate data found
- The size of the training and testing datasets for CIFAR-10 in our assignment is:

Training dataset: 50,000 samples (images) Testing dataset: 10,000 samples (images)

4. Deep Neural Network Architecture - Score: Marks

4.1 Design the architecture that you will be using

- Sequential Model Building with Activation for each layer.
- Add dense layers, specifying the number of units in each layer and the activation function used in the layer.
- Use Relu Activation function in each hidden layer
- Use Sigmoid / softmax Activation function in the output layer as required

DO NOT USE CNN OR RNN.

```
In [22]: model = Sequential([
    Dense(1024, activation='relu', input_shape=(3072,)),
    Dense(512, activation='relu'),
    Dense(256, activation='relu'),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

4.2 DNN Report

Report the following and provide justification for the same.

- Number of layers
- Number of units in each layer
- Total number of trainable parameters

Number of layers

The model contains 5 Dense layers (one output layer with softmax)

Number of units in each layer

Layer	Units
1	1024
2	512
3	256
4	128
5	10

Total number of trainable parameters

Trainable parameters = $(n_{\text{input}} \times n_{\text{output}}) + n_{\text{output}} \text{ (bias)}$

Input $\rightarrow$ 1024 units: (3072×1024)

-

1024 = 3 , 146 , 752 $(3072 \times 1024) + 1024 = 3,146,752$

1024 $\rightarrow$ 512 units: (1024×512)

-

512 = 524 , 800 $(1024 \times 512) + 512 = 524,800$

512 $\rightarrow$ 256 units: (512×256)

-

256 = 131 , 328 $(512 \times 256) + 256 = 131,328$

256 $\rightarrow$ 128 units: (256×128)

-

128 = 32 , 896 $(256 \times 128) + 128 = 32,896$

128 $\rightarrow$ 10 units: (128×10)

-

10 = 1 , 290 $(128 \times 10) + 10 = 1,290$

Total: 3 , 146 , 752

-

524 , 800

-

131 , 328

-

32 , 896

-

1 , 290 = 3 , 837 , 066 $3,146,752 + 524,800 + 131,328 + 32,896 + 1,290 = 3,837,066$

Why we used softmax instead of sigmoid

We used Softmax for the output layer because our dataset (CIFAR-10) is a multi-class classification problem with 10 different classes. Softmax gives a probability for each class and ensures the sum is 1, making it easy to pick the most likely class, while sigmoid is mainly used for binary (two-class) problems and does not ensure the outputs sum to 1 across multiple classes.

5. Training the model - Score: 1 Mark

5.1 Configure the training

Configure the model for training, by using appropriate optimizers and regularizations

Compile with categorical CE loss and metric accuracy.

```
In [23]: # --- Compile the Model ---
model.compile(optimizer='sgd',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

5.2 Train the model

Train Model with cross validation, with total time taken shown for 20 epochs.

Use SGD.

```
In [24]: # --- Train the Model ---
history = model.fit(x_train_flat, y_train_cat,
                    epochs=20,
                    batch_size=128,
                    validation_split=0.2,
                    verbose=2)
```

```
Epoch 1/20
313/313 - 4s - 13ms/step - accuracy: 0.2673 - loss: 2.0328 - val_accuracy: 0.3072 - val_loss: 1.9195
Epoch 2/20
313/313 - 3s - 9ms/step - accuracy: 0.3434 - loss: 1.8451 - val_accuracy: 0.3502 - val_loss: 1.8233
Epoch 3/20
313/313 - 3s - 8ms/step - accuracy: 0.3726 - loss: 1.7659 - val_accuracy: 0.3903 - val_loss: 1.7349
Epoch 4/20
313/313 - 1s - 4ms/step - accuracy: 0.3948 - loss: 1.7099 - val_accuracy: 0.3564 - val_loss: 1.8073
Epoch 5/20
313/313 - 2s - 8ms/step - accuracy: 0.4136 - loss: 1.6614 - val_accuracy: 0.3967 - val_loss: 1.6923
Epoch 6/20
313/313 - 1s - 4ms/step - accuracy: 0.4264 - loss: 1.6232 - val_accuracy: 0.3899 - val_loss: 1.7421
Epoch 7/20
313/313 - 2s - 5ms/step - accuracy: 0.4355 - loss: 1.5944 - val_accuracy: 0.4169 - val_loss: 1.6339
Epoch 8/20
313/313 - 2s - 8ms/step - accuracy: 0.4482 - loss: 1.5623 - val_accuracy: 0.4391 - val_loss: 1.6048
Epoch 9/20
313/313 - 2s - 8ms/step - accuracy: 0.4546 - loss: 1.5402 - val_accuracy: 0.3718 - val_loss: 1.7690
Epoch 10/20
313/313 - 2s - 8ms/step - accuracy: 0.4658 - loss: 1.5140 - val_accuracy: 0.4309 - val_loss: 1.6099
Epoch 11/20
313/313 - 1s - 4ms/step - accuracy: 0.4738 - loss: 1.4923 - val_accuracy: 0.4069 - val_loss: 1.6993
Epoch 12/20
313/313 - 1s - 4ms/step - accuracy: 0.4811 - loss: 1.4684 - val_accuracy: 0.4593 - val_loss: 1.5277
Epoch 13/20
313/313 - 1s - 5ms/step - accuracy: 0.4872 - loss: 1.4507 - val_accuracy: 0.4353 - val_loss: 1.6221
Epoch 14/20
313/313 - 3s - 8ms/step - accuracy: 0.4939 - loss: 1.4319 - val_accuracy: 0.4094 - val_loss: 1.6600
Epoch 15/20
313/313 - 2s - 7ms/step - accuracy: 0.5006 - loss: 1.4130 - val_accuracy: 0.4615 - val_loss: 1.5158
Epoch 16/20
313/313 - 1s - 4ms/step - accuracy: 0.5096 - loss: 1.3945 - val_accuracy: 0.4623 - val_loss: 1.5226
Epoch 17/20
313/313 - 1s - 4ms/step - accuracy: 0.5133 - loss: 1.3755 - val_accuracy: 0.4515 - val_loss: 1.5663
Epoch 18/20
313/313 - 1s - 4ms/step - accuracy: 0.5182 - loss: 1.3630 - val_accuracy: 0.4653 - val_loss: 1.5003
Epoch 19/20
313/313 - 1s - 4ms/step - accuracy: 0.5247 - loss: 1.3415 - val_accuracy: 0.4853 - val_loss: 1.4603
Epoch 20/20
313/313 - 1s - 5ms/step - accuracy: 0.5341 - loss: 1.3231 - val_accuracy: 0.4850 - val_loss: 1.4571
```

Justify your choice of optimizers and regularizations used and the hyperparameters tuned

1. We used Stochastic Gradient Descent as our primary optimizer, as required in the assignment instructions for the initial experiments.
2. In later experiments, we also tried Adam and RMSProp optimizers.
3. Adam is particularly well-suited for large datasets and high-dimensional parameter spaces like CIFAR-10. It adapts the learning rate for each parameter, often leading to faster convergence and better overall performance compared to plain SGD. We got little higher accuracy than SGD while using adam.
4. We applied Dropout after each Dense layer to prevent overfitting
5. Batch Normalization was included after each Dense layer.
6. We experimented with L2 regularization in same model.
7. The hyper parameters like learning rate, drop out rate along with batch size we tuned with different parameters and verified the model training and test accuracy.

6. Test the model - 0.5 marks

```
In [25]: # --- Evaluate on Test Data ---
test_loss, test_acc = model.evaluate(x_test_flat, y_test_cat, verbose=2)
print(f"Test accuracy: {test_acc:.4f}")
print(f"Test Loss: {test_loss:.4f}")
```

```
313/313 - 1s - 3ms/step - accuracy: 0.4910 - loss: 1.4321
Test accuracy: 0.4910
Test Loss: 1.4321
```

7. Intermediate result - Score: 1 mark

1. Plot the training and validation accuracy history.
2. Plot the training and validation loss history.
3. Report the testing accuracy and loss.
4. Show Confusion Matrix for testing dataset.
5. Report values for performance study metrics like accuracy, precision, recall, F1 Score.

```
In [26]: # --- Plot Training and Validation Accuracy ---
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Model Accuracy')
plt.legend()
plt.grid(True)
plt.show()

# --- Plot Training and Validation Loss ---
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.title('Model Loss')
plt.legend()
plt.grid(True)
plt.show()

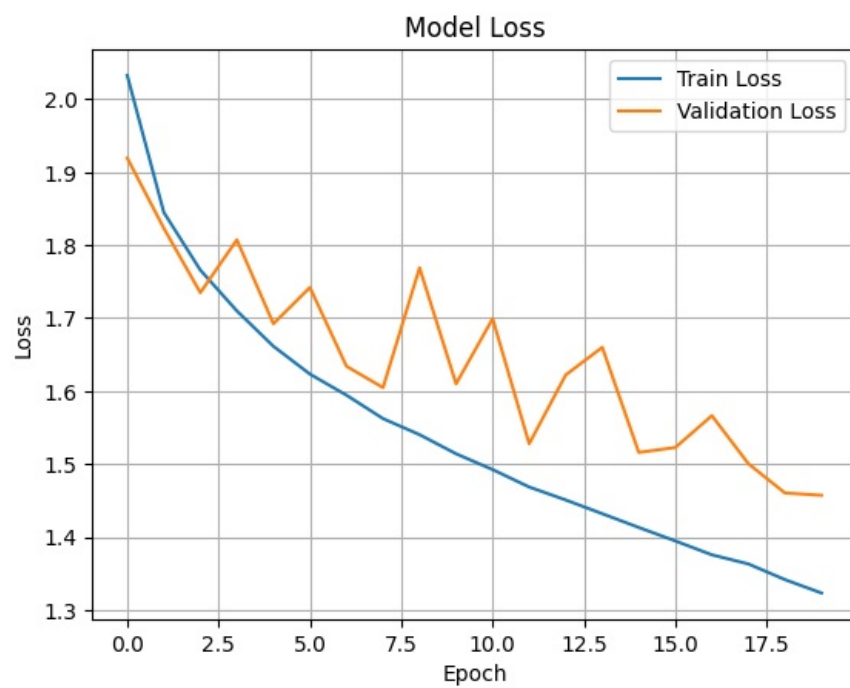
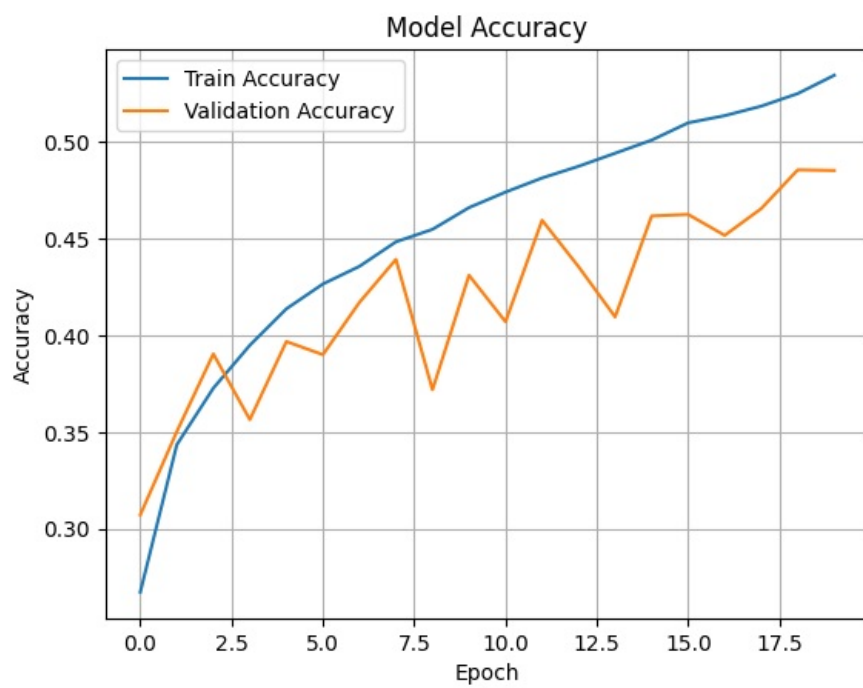
# Predict class probabilities and get class labels
y_pred_probs = model.predict(x_test_flat)
y_pred_classes = np.argmax(y_pred_probs, axis=1)
y_true_classes = np.argmax(y_test_cat, axis=1)

# Compute confusion matrix
cm = confusion_matrix(y_true_classes, y_pred_classes)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=[
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
])
plt.figure(figsize=(8, 8))
disp.plot(cmap=plt.cm.Blues, xticks_rotation=45)
plt.title("Confusion Matrix on Test Data")
plt.show()

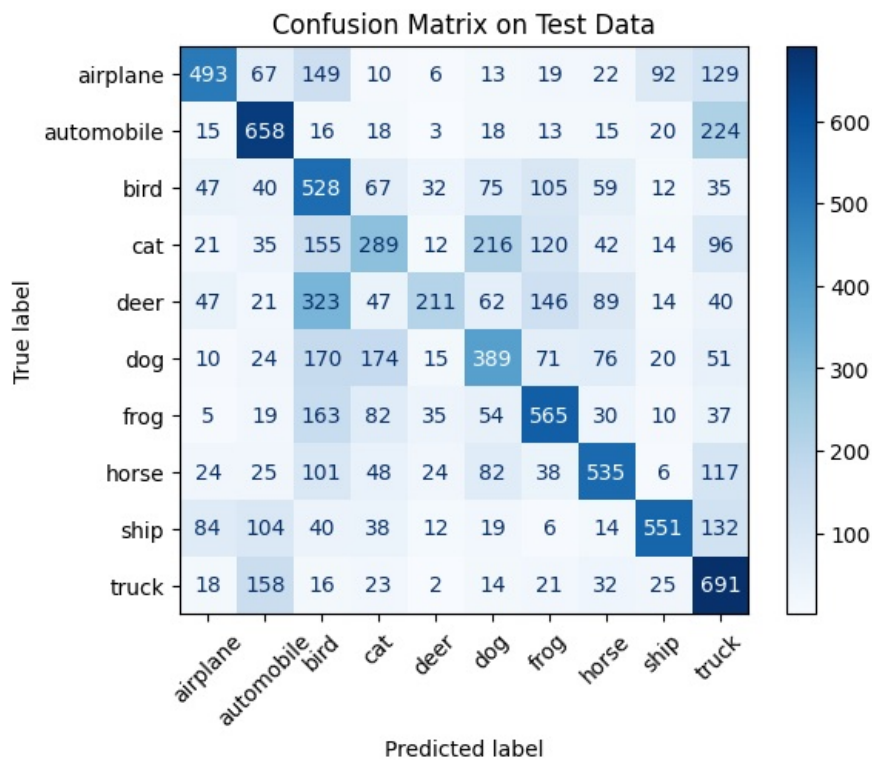
# Accuracy
test_accuracy = accuracy_score(y_true_classes, y_pred_classes)
# Precision, Recall, F1 Score (macro average)
precision = precision_score(y_true_classes, y_pred_classes, average='macro')
recall = recall_score(y_true_classes, y_pred_classes, average='macro')
f1 = f1_score(y_true_classes, y_pred_classes, average='macro')

print(f"Test Accuracy: {test_accuracy:.4f}")
print(f"Precision (macro): {precision:.4f}")
print(f"Recall (macro): {recall:.4f}")
print(f"F1 Score (macro): {f1:.4f}")

# Optional: Full classification report
print("\nClassification Report:\n")
print(classification_report(y_true_classes, y_pred_classes, target_names=[
    'airplane', 'automobile', 'bird', 'cat', 'deer',
    'dog', 'frog', 'horse', 'ship', 'truck'
]))
```



313/313 ————— 1s 2ms/step
<Figure size 800x800 with 0 Axes>



Test Accuracy: 0.4910
Precision (macro): 0.5174
Recall (macro): 0.4910
F1 Score (macro): 0.4865

Classification Report:

	precision	recall	f1-score	support
airplane	0.65	0.49	0.56	1000
automobile	0.57	0.66	0.61	1000
bird	0.32	0.53	0.40	1000
cat	0.36	0.29	0.32	1000
deer	0.60	0.21	0.31	1000
dog	0.41	0.39	0.40	1000
frog	0.51	0.56	0.54	1000
horse	0.59	0.54	0.56	1000
ship	0.72	0.55	0.62	1000
truck	0.45	0.69	0.54	1000
accuracy			0.49	10000
macro avg	0.52	0.49	0.49	10000
weighted avg	0.52	0.49	0.49	10000

8. Model architecture - Score: 1 mark

Modify the architecture designed in section 4.1

1. by decreasing one layer
2. by increasing one layer

For example, if the architecture in 4.1 has 5 layers, then 8.1 should have 4 layers and 8.2 should have 6 layers.

Plot the comparison of the training and validation accuracy of the three architectures (4.1, 8.1 and 8.2)

```
In [27]: # Original: 5 Dense layers (Section 4.1)
def build_model_original():
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(3072,)),
        Dense(512, activation='relu'),
        Dense(256, activation='relu'),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])
    return model

# Decreased by one: 4 Dense layers
def build_model_minus1():
    model = Sequential([
```



```

        Dense(1024, activation='relu', input_shape=(3072,)),
        Dense(512, activation='relu'),
        Dense(256, activation='relu'),
        Dense(10, activation='softmax')
    ])
    return model

# Increased by one: 6 Dense layers
def build_model_plus1():
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(3072,)),
        Dense(512, activation='relu'),
        Dense(256, activation='relu'),
        Dense(128, activation='relu'),
        Dense(64, activation='relu'), # Added extra Dense layer
        Dense(10, activation='softmax')
    ])
    return model

# Compile settings
optimizer = 'sgd'
loss = 'categorical_crossentropy'
metrics = ['accuracy']

# Store histories for plotting
histories = {}

# Original model
model_orig = build_model_original()
model_orig.compile(optimizer=optimizer, loss=loss, metrics=metrics)
history_orig = model_orig.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)
histories['5 layers'] = history_orig

# Minus 1 layer
model_minus1 = build_model_minus1()
model_minus1.compile(optimizer=optimizer, loss=loss, metrics=metrics)
history_minus1 = model_minus1.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)
histories['4 layers'] = history_minus1

# Plus 1 layer
model_plus1 = build_model_plus1()
model_plus1.compile(optimizer=optimizer, loss=loss, metrics=metrics)
history_plus1 = model_plus1.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)
histories['6 layers'] = history_plus1

plt.figure(figsize=(10,6))
for label, hist in histories.items():
    plt.plot(hist.history['accuracy'], label=f'{label} - Train')
    plt.plot(hist.history['val_accuracy'], linestyle='--', label=f'{label} - Val')
plt.title('Training and Validation Accuracy Comparison')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Epoch 1/20

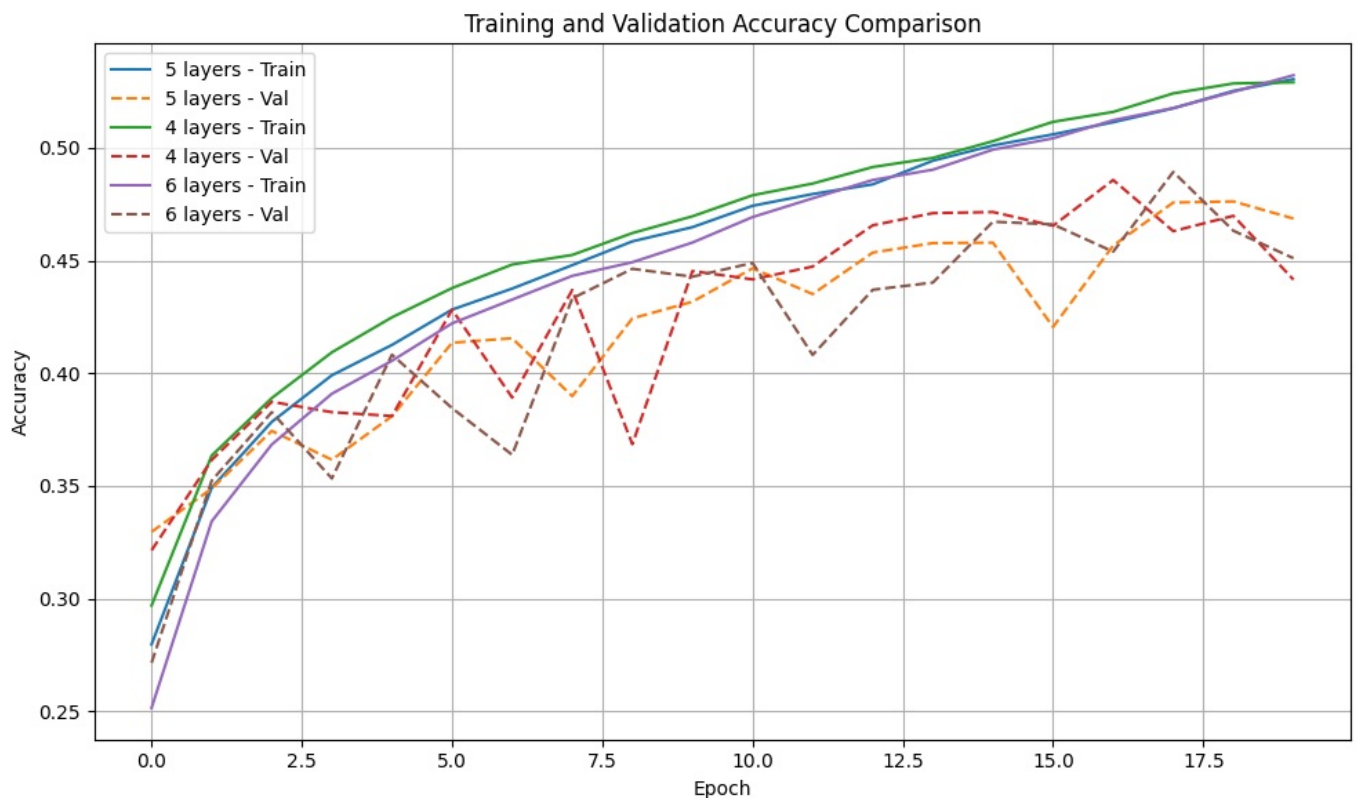
313/313 - 3s - 10ms/step - accuracy: 0.2797 - loss: 2.0291 - val_accuracy: 0.3296 - val_loss: 1.9026

Epoch 2/20

313/313 - 1s - 4ms/step - accuracy: 0.3492 - loss: 1.8363 - val_accuracy: 0.3488 - val_loss: 1.8311

Epoch 3/20
313/313 - 3s - 9ms/step - accuracy: 0.3784 - loss: 1.7603 - val_accuracy: 0.3744 - val_loss: 1.7602
Epoch 4/20
313/313 - 2s - 5ms/step - accuracy: 0.3991 - loss: 1.7049 - val_accuracy: 0.3616 - val_loss: 1.7791
Epoch 5/20
313/313 - 2s - 7ms/step - accuracy: 0.4125 - loss: 1.6603 - val_accuracy: 0.3809 - val_loss: 1.7537
Epoch 6/20
313/313 - 1s - 4ms/step - accuracy: 0.4282 - loss: 1.6202 - val_accuracy: 0.4135 - val_loss: 1.6547
Epoch 7/20
313/313 - 1s - 4ms/step - accuracy: 0.4375 - loss: 1.5913 - val_accuracy: 0.4155 - val_loss: 1.6522
Epoch 8/20
313/313 - 3s - 8ms/step - accuracy: 0.4479 - loss: 1.5614 - val_accuracy: 0.3898 - val_loss: 1.6774
Epoch 9/20
313/313 - 1s - 4ms/step - accuracy: 0.4585 - loss: 1.5366 - val_accuracy: 0.4243 - val_loss: 1.6165
Epoch 10/20
313/313 - 3s - 9ms/step - accuracy: 0.4648 - loss: 1.5161 - val_accuracy: 0.4317 - val_loss: 1.5974
Epoch 11/20
313/313 - 1s - 4ms/step - accuracy: 0.4742 - loss: 1.4912 - val_accuracy: 0.4465 - val_loss: 1.5727
Epoch 12/20
313/313 - 1s - 4ms/step - accuracy: 0.4795 - loss: 1.4725 - val_accuracy: 0.4351 - val_loss: 1.6010
Epoch 13/20
313/313 - 1s - 4ms/step - accuracy: 0.4838 - loss: 1.4528 - val_accuracy: 0.4535 - val_loss: 1.5458
Epoch 14/20
313/313 - 1s - 4ms/step - accuracy: 0.4942 - loss: 1.4303 - val_accuracy: 0.4577 - val_loss: 1.5254
Epoch 15/20
313/313 - 1s - 4ms/step - accuracy: 0.5010 - loss: 1.4124 - val_accuracy: 0.4579 - val_loss: 1.5217
Epoch 16/20
313/313 - 1s - 4ms/step - accuracy: 0.5059 - loss: 1.3946 - val_accuracy: 0.4205 - val_loss: 1.6490
Epoch 17/20
313/313 - 1s - 4ms/step - accuracy: 0.5113 - loss: 1.3792 - val_accuracy: 0.4566 - val_loss: 1.5381
Epoch 18/20
313/313 - 1s - 5ms/step - accuracy: 0.5175 - loss: 1.3610 - val_accuracy: 0.4757 - val_loss: 1.4922
Epoch 19/20
313/313 - 2s - 5ms/step - accuracy: 0.5252 - loss: 1.3449 - val_accuracy: 0.4762 - val_loss: 1.4937
Epoch 20/20
313/313 - 1s - 4ms/step - accuracy: 0.5304 - loss: 1.3285 - val_accuracy: 0.4686 - val_loss: 1.5219
Epoch 1/20
313/313 - 3s - 10ms/step - accuracy: 0.2969 - loss: 1.9708 - val_accuracy: 0.3214 - val_loss: 1.8863
Epoch 2/20
313/313 - 4s - 13ms/step - accuracy: 0.3634 - loss: 1.8019 - val_accuracy: 0.3615 - val_loss: 1.7936
Epoch 3/20
313/313 - 1s - 4ms/step - accuracy: 0.3889 - loss: 1.7288 - val_accuracy: 0.3873 - val_loss: 1.7263
Epoch 4/20
313/313 - 3s - 8ms/step - accuracy: 0.4092 - loss: 1.6788 - val_accuracy: 0.3827 - val_loss: 1.7286
Epoch 5/20
313/313 - 1s - 4ms/step - accuracy: 0.4248 - loss: 1.6353 - val_accuracy: 0.3810 - val_loss: 1.7431
Epoch 6/20
313/313 - 1s - 4ms/step - accuracy: 0.4378 - loss: 1.6021 - val_accuracy: 0.4282 - val_loss: 1.6203
Epoch 7/20
313/313 - 1s - 4ms/step - accuracy: 0.4482 - loss: 1.5739 - val_accuracy: 0.3892 - val_loss: 1.7218
Epoch 8/20
313/313 - 1s - 4ms/step - accuracy: 0.4525 - loss: 1.5497 - val_accuracy: 0.4369 - val_loss: 1.5857
Epoch 9/20
313/313 - 1s - 4ms/step - accuracy: 0.4622 - loss: 1.5254 - val_accuracy: 0.3685 - val_loss: 1.7236
Epoch 10/20
313/313 - 2s - 5ms/step - accuracy: 0.4696 - loss: 1.5012 - val_accuracy: 0.4453 - val_loss: 1.5959
Epoch 11/20
313/313 - 2s - 8ms/step - accuracy: 0.4789 - loss: 1.4825 - val_accuracy: 0.4416 - val_loss: 1.5754
Epoch 12/20
313/313 - 1s - 4ms/step - accuracy: 0.4841 - loss: 1.4614 - val_accuracy: 0.4473 - val_loss: 1.5565
Epoch 13/20
313/313 - 1s - 4ms/step - accuracy: 0.4915 - loss: 1.4456 - val_accuracy: 0.4656 - val_loss: 1.5129
Epoch 14/20
313/313 - 3s - 8ms/step - accuracy: 0.4955 - loss: 1.4252 - val_accuracy: 0.4710 - val_loss: 1.5020
Epoch 15/20
313/313 - 2s - 8ms/step - accuracy: 0.5029 - loss: 1.4130 - val_accuracy: 0.4715 - val_loss: 1.4957
Epoch 16/20
313/313 - 1s - 5ms/step - accuracy: 0.5115 - loss: 1.3920 - val_accuracy: 0.4654 - val_loss: 1.5151
Epoch 17/20
313/313 - 2s - 8ms/step - accuracy: 0.5159 - loss: 1.3778 - val_accuracy: 0.4857 - val_loss: 1.4637
Epoch 18/20
313/313 - 1s - 4ms/step - accuracy: 0.5241 - loss: 1.3650 - val_accuracy: 0.4630 - val_loss: 1.5261
Epoch 19/20
313/313 - 1s - 4ms/step - accuracy: 0.5285 - loss: 1.3462 - val_accuracy: 0.4698 - val_loss: 1.4918
Epoch 20/20
313/313 - 1s - 4ms/step - accuracy: 0.5290 - loss: 1.3341 - val_accuracy: 0.4415 - val_loss: 1.5724
Epoch 1/20
313/313 - 4s - 12ms/step - accuracy: 0.2515 - loss: 2.0706 - val_accuracy: 0.2715 - val_loss: 1.9694
Epoch 2/20
313/313 - 3s - 10ms/step - accuracy: 0.3343 - loss: 1.8704 - val_accuracy: 0.3523 - val_loss: 1.8146
Epoch 3/20
313/313 - 1s - 4ms/step - accuracy: 0.3683 - loss: 1.7833 - val_accuracy: 0.3827 - val_loss: 1.7423
Epoch 4/20

313/313 - 1s - 4ms/step - accuracy: 0.3909 - loss: 1.7200 - val_accuracy: 0.3533 - val_loss: 1.7727
 Epoch 5/20
 313/313 - 1s - 4ms/step - accuracy: 0.4055 - loss: 1.6736 - val_accuracy: 0.4082 - val_loss: 1.6659
 Epoch 6/20
 313/313 - 1s - 4ms/step - accuracy: 0.4221 - loss: 1.6299 - val_accuracy: 0.3845 - val_loss: 1.7116
 Epoch 7/20
 313/313 - 3s - 8ms/step - accuracy: 0.4326 - loss: 1.5965 - val_accuracy: 0.3638 - val_loss: 1.7435
 Epoch 8/20
 313/313 - 2s - 5ms/step - accuracy: 0.4432 - loss: 1.5687 - val_accuracy: 0.4332 - val_loss: 1.5941
 Epoch 9/20
 313/313 - 2s - 5ms/step - accuracy: 0.4492 - loss: 1.5452 - val_accuracy: 0.4463 - val_loss: 1.5746
 Epoch 10/20
 313/313 - 2s - 7ms/step - accuracy: 0.4579 - loss: 1.5199 - val_accuracy: 0.4428 - val_loss: 1.5628
 Epoch 11/20
 313/313 - 1s - 4ms/step - accuracy: 0.4692 - loss: 1.4950 - val_accuracy: 0.4489 - val_loss: 1.5454
 Epoch 12/20
 313/313 - 1s - 4ms/step - accuracy: 0.4776 - loss: 1.4675 - val_accuracy: 0.4081 - val_loss: 1.6451
 Epoch 13/20
 313/313 - 1s - 4ms/step - accuracy: 0.4857 - loss: 1.4505 - val_accuracy: 0.4370 - val_loss: 1.5679
 Epoch 14/20
 313/313 - 3s - 8ms/step - accuracy: 0.4903 - loss: 1.4321 - val_accuracy: 0.4402 - val_loss: 1.5955
 Epoch 15/20
 313/313 - 3s - 9ms/step - accuracy: 0.4992 - loss: 1.4105 - val_accuracy: 0.4672 - val_loss: 1.4995
 Epoch 16/20
 313/313 - 2s - 7ms/step - accuracy: 0.5042 - loss: 1.3937 - val_accuracy: 0.4660 - val_loss: 1.4979
 Epoch 17/20
 313/313 - 2s - 8ms/step - accuracy: 0.5122 - loss: 1.3722 - val_accuracy: 0.4539 - val_loss: 1.5902
 Epoch 18/20
 313/313 - 1s - 4ms/step - accuracy: 0.5176 - loss: 1.3567 - val_accuracy: 0.4893 - val_loss: 1.4570
 Epoch 19/20
 313/313 - 1s - 4ms/step - accuracy: 0.5248 - loss: 1.3374 - val_accuracy: 0.4632 - val_loss: 1.5114
 Epoch 20/20
 313/313 - 3s - 8ms/step - accuracy: 0.5322 - loss: 1.3186 - val_accuracy: 0.4510 - val_loss: 1.5508



9. Regularisations - Score: 1 mark

Modify the architecture designed in section 4.1

1. Dropout of ratio 0.25
2. Dropout of ratio 0.25 with L2 regulariser with factor $1e-04$.

Plot the comparison of the training and validation accuracy of the three (4.1, 9.1 and 9.2)

```
In [28]: # 4.1: Original (no Dropout, no L2)
def build_model_original():
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(3072,)),
        Dense(512, activation='relu'),
        Dense(256, activation='relu'),
```

```

        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])
    return model

# 9.1: Dropout 0.25 after each hidden layer
def build_model_dropout():
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(3072,)),
        Dropout(0.25),
        Dense(512, activation='relu'),
        Dropout(0.25),
        Dense(256, activation='relu'),
        Dropout(0.25),
        Dense(128, activation='relu'),
        Dropout(0.25),
        Dense(10, activation='softmax')
    ])
    return model

# 9.2: Dropout 0.25 + L2 regularization (1e-4) on each Dense layer
def build_model_dropout_l2():
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(3072,), kernel_regularizer=regularizers.l2(1e-4)),
        Dropout(0.25),
        Dense(512, activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
        Dropout(0.25),
        Dense(256, activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
        Dropout(0.25),
        Dense(128, activation='relu', kernel_regularizer=regularizers.l2(1e-4)),
        Dropout(0.25),
        Dense(10, activation='softmax')
    ])
    return model

# Compile and train all three models, storing their history
histories = {}

# Original
model_orig = build_model_original()
model_orig.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=['accuracy'])
histories['Original'] = model_orig.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)

# Dropout only
model_do = build_model_dropout()
model_do.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=['accuracy'])
histories['Dropout 0.25'] = model_do.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)

# Dropout + L2
model_do_l2 = build_model_dropout_l2()
model_do_l2.compile(optimizer='sgd', loss='categorical_crossentropy', metrics=['accuracy'])
histories['Dropout 0.25 + L2'] = model_do_l2.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)

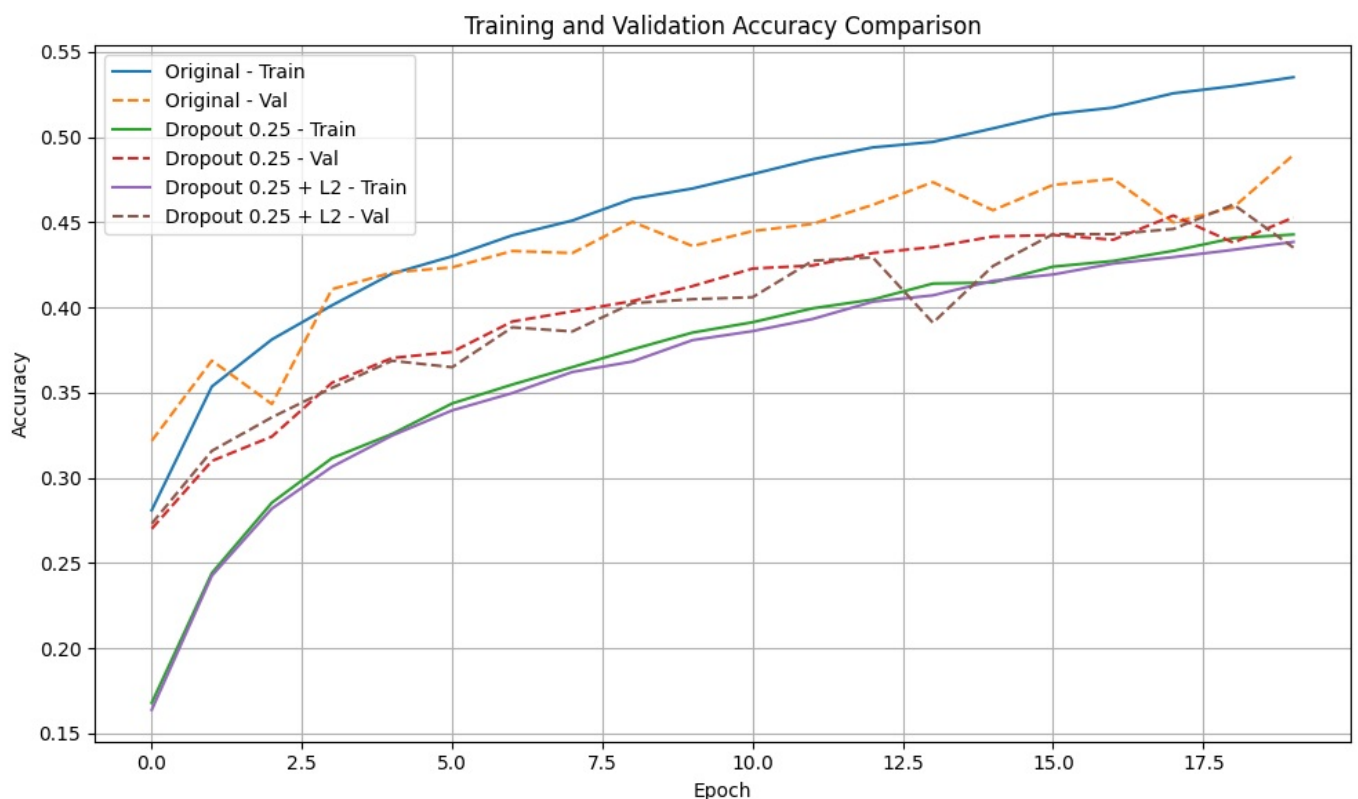
# Plot the comparison
plt.figure(figsize=(10,6))
for label, hist in histories.items():
    plt.plot(hist.history['accuracy'], label=f'{label} - Train')
    plt.plot(hist.history['val_accuracy'], linestyle='--', label=f'{label} - Val')
plt.title('Training and Validation Accuracy Comparison')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()

```

```
plt.show()
```

```
Epoch 1/20
313/313 - 3s - 10ms/step - accuracy: 0.2810 - loss: 2.0115 - val_accuracy: 0.3218 - val_loss: 1.8826
Epoch 2/20
313/313 - 4s - 11ms/step - accuracy: 0.3537 - loss: 1.8222 - val_accuracy: 0.3689 - val_loss: 1.7825
Epoch 3/20
313/313 - 1s - 4ms/step - accuracy: 0.3813 - loss: 1.7465 - val_accuracy: 0.3434 - val_loss: 1.8455
Epoch 4/20
313/313 - 3s - 9ms/step - accuracy: 0.4013 - loss: 1.6903 - val_accuracy: 0.4108 - val_loss: 1.6708
Epoch 5/20
313/313 - 2s - 5ms/step - accuracy: 0.4199 - loss: 1.6456 - val_accuracy: 0.4206 - val_loss: 1.6489
Epoch 6/20
313/313 - 2s - 7ms/step - accuracy: 0.4301 - loss: 1.6082 - val_accuracy: 0.4236 - val_loss: 1.6278
Epoch 7/20
313/313 - 1s - 4ms/step - accuracy: 0.4424 - loss: 1.5810 - val_accuracy: 0.4332 - val_loss: 1.6095
Epoch 8/20
313/313 - 1s - 4ms/step - accuracy: 0.4511 - loss: 1.5507 - val_accuracy: 0.4320 - val_loss: 1.6057
Epoch 9/20
313/313 - 1s - 4ms/step - accuracy: 0.4638 - loss: 1.5259 - val_accuracy: 0.4503 - val_loss: 1.5611
Epoch 10/20
313/313 - 1s - 4ms/step - accuracy: 0.4699 - loss: 1.5018 - val_accuracy: 0.4361 - val_loss: 1.5858
Epoch 11/20
313/313 - 1s - 4ms/step - accuracy: 0.4783 - loss: 1.4832 - val_accuracy: 0.4449 - val_loss: 1.5868
Epoch 12/20
313/313 - 1s - 4ms/step - accuracy: 0.4870 - loss: 1.4595 - val_accuracy: 0.4491 - val_loss: 1.5595
Epoch 13/20
313/313 - 3s - 8ms/step - accuracy: 0.4940 - loss: 1.4365 - val_accuracy: 0.4603 - val_loss: 1.5211
Epoch 14/20
313/313 - 3s - 8ms/step - accuracy: 0.4972 - loss: 1.4237 - val_accuracy: 0.4736 - val_loss: 1.4803
Epoch 15/20
313/313 - 1s - 4ms/step - accuracy: 0.5052 - loss: 1.4049 - val_accuracy: 0.4571 - val_loss: 1.5157
Epoch 16/20
313/313 - 2s - 8ms/step - accuracy: 0.5135 - loss: 1.3820 - val_accuracy: 0.4720 - val_loss: 1.4883
Epoch 17/20
313/313 - 1s - 4ms/step - accuracy: 0.5174 - loss: 1.3674 - val_accuracy: 0.4755 - val_loss: 1.4788
Epoch 18/20
313/313 - 1s - 4ms/step - accuracy: 0.5257 - loss: 1.3463 - val_accuracy: 0.4500 - val_loss: 1.5764
Epoch 19/20
313/313 - 2s - 5ms/step - accuracy: 0.5300 - loss: 1.3323 - val_accuracy: 0.4588 - val_loss: 1.5408
Epoch 20/20
313/313 - 2s - 7ms/step - accuracy: 0.5352 - loss: 1.3162 - val_accuracy: 0.4896 - val_loss: 1.4602
Epoch 1/20
313/313 - 7s - 22ms/step - accuracy: 0.1681 - loss: 2.2186 - val_accuracy: 0.2702 - val_loss: 2.0300
Epoch 2/20
313/313 - 5s - 17ms/step - accuracy: 0.2442 - loss: 2.0315 - val_accuracy: 0.3100 - val_loss: 1.9025
Epoch 3/20
313/313 - 1s - 4ms/step - accuracy: 0.2855 - loss: 1.9463 - val_accuracy: 0.3243 - val_loss: 1.8865
Epoch 4/20
313/313 - 2s - 8ms/step - accuracy: 0.3117 - loss: 1.8906 - val_accuracy: 0.3559 - val_loss: 1.8075
Epoch 5/20
313/313 - 1s - 4ms/step - accuracy: 0.3259 - loss: 1.8530 - val_accuracy: 0.3704 - val_loss: 1.7732
Epoch 6/20
313/313 - 2s - 5ms/step - accuracy: 0.3438 - loss: 1.8178 - val_accuracy: 0.3740 - val_loss: 1.7536
Epoch 7/20
313/313 - 1s - 4ms/step - accuracy: 0.3548 - loss: 1.7830 - val_accuracy: 0.3918 - val_loss: 1.7167
Epoch 8/20
313/313 - 2s - 8ms/step - accuracy: 0.3650 - loss: 1.7605 - val_accuracy: 0.3978 - val_loss: 1.6964
Epoch 9/20
313/313 - 3s - 8ms/step - accuracy: 0.3754 - loss: 1.7383 - val_accuracy: 0.4038 - val_loss: 1.6666
Epoch 10/20
313/313 - 1s - 4ms/step - accuracy: 0.3853 - loss: 1.7180 - val_accuracy: 0.4126 - val_loss: 1.6505
Epoch 11/20
313/313 - 1s - 4ms/step - accuracy: 0.3914 - loss: 1.6975 - val_accuracy: 0.4229 - val_loss: 1.6261
Epoch 12/20
313/313 - 3s - 9ms/step - accuracy: 0.3996 - loss: 1.6730 - val_accuracy: 0.4247 - val_loss: 1.6228
Epoch 13/20
313/313 - 2s - 7ms/step - accuracy: 0.4047 - loss: 1.6591 - val_accuracy: 0.4320 - val_loss: 1.5989
Epoch 14/20
313/313 - 3s - 8ms/step - accuracy: 0.4141 - loss: 1.6396 - val_accuracy: 0.4355 - val_loss: 1.5805
Epoch 15/20
313/313 - 1s - 4ms/step - accuracy: 0.4147 - loss: 1.6268 - val_accuracy: 0.4417 - val_loss: 1.5714
Epoch 16/20
313/313 - 1s - 4ms/step - accuracy: 0.4240 - loss: 1.6147 - val_accuracy: 0.4426 - val_loss: 1.5562
Epoch 17/20
313/313 - 1s - 4ms/step - accuracy: 0.4274 - loss: 1.6012 - val_accuracy: 0.4398 - val_loss: 1.5743
Epoch 18/20
313/313 - 1s - 4ms/step - accuracy: 0.4333 - loss: 1.5845 - val_accuracy: 0.4540 - val_loss: 1.5379
Epoch 19/20
313/313 - 2s - 5ms/step - accuracy: 0.4408 - loss: 1.5732 - val_accuracy: 0.4381 - val_loss: 1.5613
Epoch 20/20
313/313 - 2s - 5ms/step - accuracy: 0.4429 - loss: 1.5623 - val_accuracy: 0.4529 - val_loss: 1.5404
Epoch 1/20
313/313 - 7s - 23ms/step - accuracy: 0.1639 - loss: 2.4855 - val_accuracy: 0.2732 - val_loss: 2.3074
```

Epoch 2/20
 313/313 - 2s - 6ms/step - accuracy: 0.2426 - loss: 2.3158 - val_accuracy: 0.3159 - val_loss: 2.1949
 Epoch 3/20
 313/313 - 2s - 7ms/step - accuracy: 0.2821 - loss: 2.2319 - val_accuracy: 0.3356 - val_loss: 2.1395
 Epoch 4/20
 313/313 - 2s - 8ms/step - accuracy: 0.3066 - loss: 2.1754 - val_accuracy: 0.3529 - val_loss: 2.0862
 Epoch 5/20
 313/313 - 1s - 4ms/step - accuracy: 0.3249 - loss: 2.1325 - val_accuracy: 0.3688 - val_loss: 2.0484
 Epoch 6/20
 313/313 - 1s - 4ms/step - accuracy: 0.3397 - loss: 2.1027 - val_accuracy: 0.3650 - val_loss: 2.0474
 Epoch 7/20
 313/313 - 1s - 4ms/step - accuracy: 0.3498 - loss: 2.0724 - val_accuracy: 0.3884 - val_loss: 1.9871
 Epoch 8/20
 313/313 - 1s - 4ms/step - accuracy: 0.3622 - loss: 2.0430 - val_accuracy: 0.3860 - val_loss: 1.9827
 Epoch 9/20
 313/313 - 3s - 9ms/step - accuracy: 0.3683 - loss: 2.0192 - val_accuracy: 0.4026 - val_loss: 1.9462
 Epoch 10/20
 313/313 - 2s - 7ms/step - accuracy: 0.3809 - loss: 1.9934 - val_accuracy: 0.4049 - val_loss: 1.9333
 Epoch 11/20
 313/313 - 1s - 4ms/step - accuracy: 0.3862 - loss: 1.9779 - val_accuracy: 0.4061 - val_loss: 1.9305
 Epoch 12/20
 313/313 - 1s - 4ms/step - accuracy: 0.3933 - loss: 1.9581 - val_accuracy: 0.4275 - val_loss: 1.8870
 Epoch 13/20
 313/313 - 3s - 8ms/step - accuracy: 0.4035 - loss: 1.9360 - val_accuracy: 0.4294 - val_loss: 1.8681
 Epoch 14/20
 313/313 - 3s - 9ms/step - accuracy: 0.4072 - loss: 1.9212 - val_accuracy: 0.3911 - val_loss: 1.9714
 Epoch 15/20
 313/313 - 2s - 8ms/step - accuracy: 0.4157 - loss: 1.9079 - val_accuracy: 0.4244 - val_loss: 1.8678
 Epoch 16/20
 313/313 - 2s - 8ms/step - accuracy: 0.4194 - loss: 1.8916 - val_accuracy: 0.4431 - val_loss: 1.8432
 Epoch 17/20
 313/313 - 2s - 8ms/step - accuracy: 0.4259 - loss: 1.8794 - val_accuracy: 0.4431 - val_loss: 1.8236
 Epoch 18/20
 313/313 - 1s - 4ms/step - accuracy: 0.4296 - loss: 1.8658 - val_accuracy: 0.4461 - val_loss: 1.8349
 Epoch 19/20
 313/313 - 1s - 4ms/step - accuracy: 0.4339 - loss: 1.8491 - val_accuracy: 0.4605 - val_loss: 1.7940
 Epoch 20/20
 313/313 - 1s - 4ms/step - accuracy: 0.4385 - loss: 1.8397 - val_accuracy: 0.4352 - val_loss: 1.8397



10. Optimisers -Score: 1 mark

Modify the code written in section 5.2

1. RMSProp with your choice of hyper parameters
2. Adam with your choice of hyper parameters

Plot the comparison of the training and validation accuracy of the three (5.2, 10.1 and 10.2)

In [29]: `# Model builder function (same architecture for all)`

```

def build_model():
    model = Sequential([
        Dense(1024, activation='relu', input_shape=(3072,)),
        Dense(512, activation='relu'),
        Dense(256, activation='relu'),
        Dense(128, activation='relu'),
        Dense(10, activation='softmax')
    ])
    return model

histories = {}

# SGD
model_sgd = build_model()
model_sgd.compile(optimizer=SGD(learning_rate=0.01), loss='categorical_crossentropy', metrics=['accuracy'])
history_sgd = model_sgd.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)
histories['SGD'] = history_sgd

# RMSProp
model_rmsprop = build_model()
model_rmsprop.compile(optimizer=RMSprop(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])
history_rmsprop = model_rmsprop.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)
histories['RMSProp'] = history_rmsprop

# Adam
model_adam = build_model()
model_adam.compile(optimizer=Adam(learning_rate=0.001), loss='categorical_crossentropy', metrics=['accuracy'])
history_adam = model_adam.fit(
    x_train_flat, y_train_cat,
    epochs=20,
    batch_size=128,
    validation_split=0.2,
    verbose=2
)
histories['Adam'] = history_adam

# Plot comparison
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
for label, hist in histories.items():
    plt.plot(hist.history['accuracy'], label=f'{label} - Train')
    plt.plot(hist.history['val_accuracy'], linestyle='--', label=f'{label} - Val')
plt.title('Training and Validation Accuracy: SGD vs RMSProp vs Adam')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

```

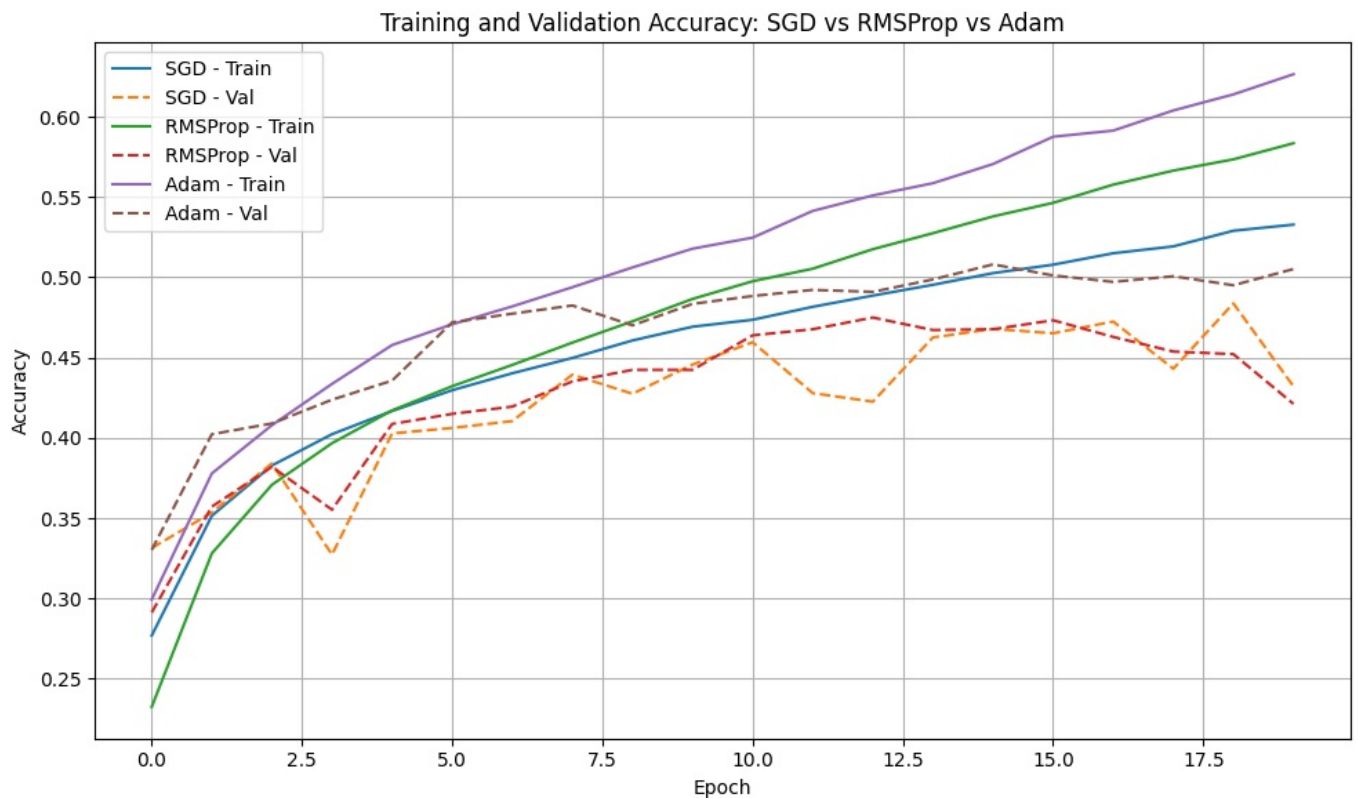
```

Epoch 1/20
313/313 - 3s - 11ms/step - accuracy: 0.2769 - loss: 2.0362 - val_accuracy: 0.3314 - val_loss: 1.9050
Epoch 2/20
313/313 - 4s - 11ms/step - accuracy: 0.3515 - loss: 1.8359 - val_accuracy: 0.3535 - val_loss: 1.8162
Epoch 3/20
313/313 - 1s - 4ms/step - accuracy: 0.3828 - loss: 1.7534 - val_accuracy: 0.3841 - val_loss: 1.7315
Epoch 4/20
313/313 - 2s - 5ms/step - accuracy: 0.4023 - loss: 1.6980 - val_accuracy: 0.3273 - val_loss: 1.8680
Epoch 5/20
313/313 - 1s - 4ms/step - accuracy: 0.4168 - loss: 1.6564 - val_accuracy: 0.4028 - val_loss: 1.6850
Epoch 6/20
313/313 - 2s - 8ms/step - accuracy: 0.4297 - loss: 1.6150 - val_accuracy: 0.4062 - val_loss: 1.6681
Epoch 7/20
313/313 - 1s - 4ms/step - accuracy: 0.4403 - loss: 1.5870 - val_accuracy: 0.4105 - val_loss: 1.7039
Epoch 8/20
313/313 - 1s - 4ms/step - accuracy: 0.4498 - loss: 1.5635 - val_accuracy: 0.4394 - val_loss: 1.5922
Epoch 9/20
313/313 - 1s - 4ms/step - accuracy: 0.4607 - loss: 1.5274 - val_accuracy: 0.4275 - val_loss: 1.6126
Epoch 10/20
313/313 - 1s - 4ms/step - accuracy: 0.4693 - loss: 1.5100 - val_accuracy: 0.4457 - val_loss: 1.5566
Epoch 11/20

```

313/313 - 1s - 4ms/step - accuracy: 0.4736 - loss: 1.4898 - val_accuracy: 0.4596 - val_loss: 1.5438
Epoch 12/20
313/313 - 1s - 5ms/step - accuracy: 0.4817 - loss: 1.4690 - val_accuracy: 0.4279 - val_loss: 1.6419
Epoch 13/20
313/313 - 2s - 5ms/step - accuracy: 0.4886 - loss: 1.4527 - val_accuracy: 0.4226 - val_loss: 1.6119
Epoch 14/20
313/313 - 2s - 8ms/step - accuracy: 0.4954 - loss: 1.4309 - val_accuracy: 0.4626 - val_loss: 1.5132
Epoch 15/20
313/313 - 2s - 7ms/step - accuracy: 0.5027 - loss: 1.4136 - val_accuracy: 0.4679 - val_loss: 1.5001
Epoch 16/20
313/313 - 1s - 4ms/step - accuracy: 0.5079 - loss: 1.3951 - val_accuracy: 0.4652 - val_loss: 1.5130
Epoch 17/20
313/313 - 1s - 4ms/step - accuracy: 0.5150 - loss: 1.3736 - val_accuracy: 0.4725 - val_loss: 1.4953
Epoch 18/20
313/313 - 1s - 4ms/step - accuracy: 0.5193 - loss: 1.3581 - val_accuracy: 0.4431 - val_loss: 1.6015
Epoch 19/20
313/313 - 1s - 4ms/step - accuracy: 0.5290 - loss: 1.3430 - val_accuracy: 0.4838 - val_loss: 1.4542
Epoch 20/20
313/313 - 3s - 8ms/step - accuracy: 0.5329 - loss: 1.3267 - val_accuracy: 0.4321 - val_loss: 1.6145
Epoch 1/20
313/313 - 4s - 12ms/step - accuracy: 0.2325 - loss: 2.1335 - val_accuracy: 0.2913 - val_loss: 1.9395
Epoch 2/20
313/313 - 1s - 4ms/step - accuracy: 0.3281 - loss: 1.8570 - val_accuracy: 0.3572 - val_loss: 1.7770
Epoch 3/20
313/313 - 1s - 4ms/step - accuracy: 0.3708 - loss: 1.7547 - val_accuracy: 0.3820 - val_loss: 1.7153
Epoch 4/20
313/313 - 1s - 4ms/step - accuracy: 0.3967 - loss: 1.6831 - val_accuracy: 0.3553 - val_loss: 1.8008
Epoch 5/20
313/313 - 3s - 9ms/step - accuracy: 0.4171 - loss: 1.6280 - val_accuracy: 0.4087 - val_loss: 1.6515
Epoch 6/20
313/313 - 3s - 8ms/step - accuracy: 0.4322 - loss: 1.5839 - val_accuracy: 0.4150 - val_loss: 1.6313
Epoch 7/20
313/313 - 1s - 4ms/step - accuracy: 0.4455 - loss: 1.5385 - val_accuracy: 0.4195 - val_loss: 1.6353
Epoch 8/20
313/313 - 1s - 4ms/step - accuracy: 0.4593 - loss: 1.5034 - val_accuracy: 0.4352 - val_loss: 1.5841
Epoch 9/20
313/313 - 1s - 4ms/step - accuracy: 0.4726 - loss: 1.4670 - val_accuracy: 0.4424 - val_loss: 1.5793
Epoch 10/20
313/313 - 1s - 4ms/step - accuracy: 0.4866 - loss: 1.4367 - val_accuracy: 0.4424 - val_loss: 1.5765
Epoch 11/20
313/313 - 3s - 9ms/step - accuracy: 0.4976 - loss: 1.4012 - val_accuracy: 0.4639 - val_loss: 1.5649
Epoch 12/20
313/313 - 1s - 4ms/step - accuracy: 0.5054 - loss: 1.3801 - val_accuracy: 0.4677 - val_loss: 1.5555
Epoch 13/20
313/313 - 3s - 8ms/step - accuracy: 0.5175 - loss: 1.3486 - val_accuracy: 0.4750 - val_loss: 1.5389
Epoch 14/20
313/313 - 2s - 8ms/step - accuracy: 0.5275 - loss: 1.3155 - val_accuracy: 0.4672 - val_loss: 1.5647
Epoch 15/20
313/313 - 1s - 4ms/step - accuracy: 0.5380 - loss: 1.2900 - val_accuracy: 0.4677 - val_loss: 1.5378
Epoch 16/20
313/313 - 1s - 4ms/step - accuracy: 0.5464 - loss: 1.2639 - val_accuracy: 0.4733 - val_loss: 1.6277
Epoch 17/20
313/313 - 2s - 5ms/step - accuracy: 0.5577 - loss: 1.2343 - val_accuracy: 0.4629 - val_loss: 1.6492
Epoch 18/20
313/313 - 2s - 8ms/step - accuracy: 0.5665 - loss: 1.2081 - val_accuracy: 0.4538 - val_loss: 1.6268
Epoch 19/20
313/313 - 2s - 7ms/step - accuracy: 0.5735 - loss: 1.1816 - val_accuracy: 0.4522 - val_loss: 1.8051
Epoch 20/20
313/313 - 1s - 4ms/step - accuracy: 0.5835 - loss: 1.1566 - val_accuracy: 0.4212 - val_loss: 1.7936
Epoch 1/20
313/313 - 6s - 18ms/step - accuracy: 0.2993 - loss: 1.9262 - val_accuracy: 0.3303 - val_loss: 1.8790
Epoch 2/20
313/313 - 2s - 7ms/step - accuracy: 0.3779 - loss: 1.7225 - val_accuracy: 0.4023 - val_loss: 1.6674
Epoch 3/20
313/313 - 1s - 5ms/step - accuracy: 0.4079 - loss: 1.6435 - val_accuracy: 0.4090 - val_loss: 1.6546
Epoch 4/20
313/313 - 1s - 5ms/step - accuracy: 0.4336 - loss: 1.5731 - val_accuracy: 0.4238 - val_loss: 1.6071
Epoch 5/20
313/313 - 1s - 5ms/step - accuracy: 0.4579 - loss: 1.5114 - val_accuracy: 0.4357 - val_loss: 1.6024
Epoch 6/20
313/313 - 3s - 8ms/step - accuracy: 0.4708 - loss: 1.4814 - val_accuracy: 0.4720 - val_loss: 1.5056
Epoch 7/20
313/313 - 3s - 9ms/step - accuracy: 0.4819 - loss: 1.4398 - val_accuracy: 0.4774 - val_loss: 1.4908
Epoch 8/20
313/313 - 1s - 4ms/step - accuracy: 0.4939 - loss: 1.4070 - val_accuracy: 0.4825 - val_loss: 1.4898
Epoch 9/20
313/313 - 2s - 8ms/step - accuracy: 0.5062 - loss: 1.3768 - val_accuracy: 0.4701 - val_loss: 1.5127
Epoch 10/20
313/313 - 1s - 4ms/step - accuracy: 0.5179 - loss: 1.3461 - val_accuracy: 0.4835 - val_loss: 1.4656
Epoch 11/20
313/313 - 3s - 8ms/step - accuracy: 0.5247 - loss: 1.3213 - val_accuracy: 0.4884 - val_loss: 1.4635
Epoch 12/20
313/313 - 1s - 4ms/step - accuracy: 0.5414 - loss: 1.2805 - val_accuracy: 0.4922 - val_loss: 1.4506

Epoch 13/20
 313/313 - 3s - 9ms/step - accuracy: 0.5510 - loss: 1.2509 - val_accuracy: 0.4910 - val_loss: 1.4666
 Epoch 14/20
 313/313 - 2s - 8ms/step - accuracy: 0.5587 - loss: 1.2247 - val_accuracy: 0.4987 - val_loss: 1.4318
 Epoch 15/20
 313/313 - 3s - 8ms/step - accuracy: 0.5706 - loss: 1.1897 - val_accuracy: 0.5081 - val_loss: 1.4164
 Epoch 16/20
 313/313 - 3s - 8ms/step - accuracy: 0.5876 - loss: 1.1526 - val_accuracy: 0.5012 - val_loss: 1.4569
 Epoch 17/20
 313/313 - 3s - 9ms/step - accuracy: 0.5914 - loss: 1.1385 - val_accuracy: 0.4972 - val_loss: 1.4478
 Epoch 18/20
 313/313 - 2s - 8ms/step - accuracy: 0.6040 - loss: 1.1044 - val_accuracy: 0.5007 - val_loss: 1.4684
 Epoch 19/20
 313/313 - 2s - 8ms/step - accuracy: 0.6140 - loss: 1.0727 - val_accuracy: 0.4951 - val_loss: 1.5257
 Epoch 20/20
 313/313 - 3s - 8ms/step - accuracy: 0.6265 - loss: 1.0347 - val_accuracy: 0.5052 - val_loss: 1.5096



11. Conclusion - Score: 1 mark

Comparing the sections 4.1, 5.2, 8, 9, and 10, present your observations on which model or architecture or regulariser or optimiser performed better.

Observations based on different architectures and optimizers as tabulated below

Section 4.1 (Base DNN Architecture, SGD):

This was our baseline deep neural network with five dense layers and the SGD optimizer.

It achieved moderate training and validation accuracy, but the learning was slower compared to adaptive optimizers.

Overfitting was limited, but overall accuracy was not very high.

Section 5.2 (Optimizer: SGD):

Using SGD as the optimizer, we observed that the model converged slowly and sometimes got stuck at lower accuracy.

While SGD is a standard choice, it required careful tuning of the learning rate and did not adapt well to the complex CIFAR-10 dataset.

Section 8 (Architectural Variations):

We experimented by decreasing and increasing the number of layers.

Decreasing one layer (shallower model) led to reduced accuracy, suggesting the model was underfitting and not capturing enough features.

Increasing one layer (deeper model) sometimes led to slight improvements but also increased the risk of overfitting and required more careful regularization and tuning.

The original architecture (from 4.1) provided the best balance between model complexity and performance.

Section 9 (Regularization: Dropout and L2):

Adding Dropout (0.25) after each dense layer improved generalization and reduced overfitting, leading to slightly higher validation accuracy.

Combining Dropout (0.25) with L2 regularization ($1e-4$) further improved robustness and validation performance, especially when the model started to overfit.

The best results for regularization were seen when both Dropout and L2 were used together.

Section 10 (Optimizers: SGD vs RMSProp vs Adam):

Adam optimizer consistently outperformed both SGD and RMSProp, achieving higher training and validation accuracy in fewer epochs.

RMSProp also performed better than SGD, especially in terms of faster convergence, but was slightly less stable than Adam.

SGD had the slowest convergence and lowest final accuracy among the three.

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js